ARTIFICIAL INTELLIGENCE

CSE 3201

Problems, State Spaces and Search

Problem Solving

We want:

– To automatically solve a problem

We need:

– A representation of the problem

– Algorithms that use some strategy to solve the problem defined in that representation

Problem Representation

- General:

– **State space**: Set of all possible states for a given problem is known as state space of the problem.

A problem is divided into a set of resolution steps from the initial state to the goal state.

– **Reduction to sub-problems**: a problem is arranged into a hierarchy of sub-problems.

States

- A problem is defined by its elements and their relations.
- In each instant of the resolution of a problem, those elements have specific descriptors (How to select them?) and relations.
- A **state** is a representation of those elements in a given moment.

- Two special states are defined:

– **Initial state** (starting point)

– **Final state** (goal state)

State Modification: Successor Function

- A successor function is needed to move between different states.

– *It is a transformation function on a state representation, which convert it into another state.*

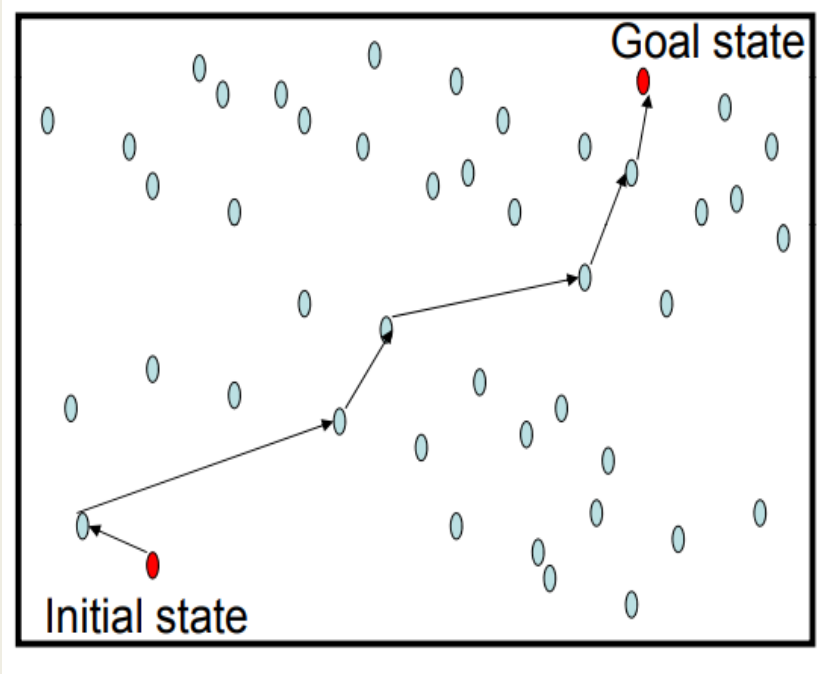
- A **successor function** is a description of possible actions, a set of operators.

State Space

- We need to formulate a state space over which we perform search.

A state space consists of:

- A set of states
  - The start state represents the initial problem
  - Each state represents some configuration reachable from the start state
  - Some states may be goal states (solutions)
- A set of operators
  -Applying an operator to a state transforms it to another state in the state space



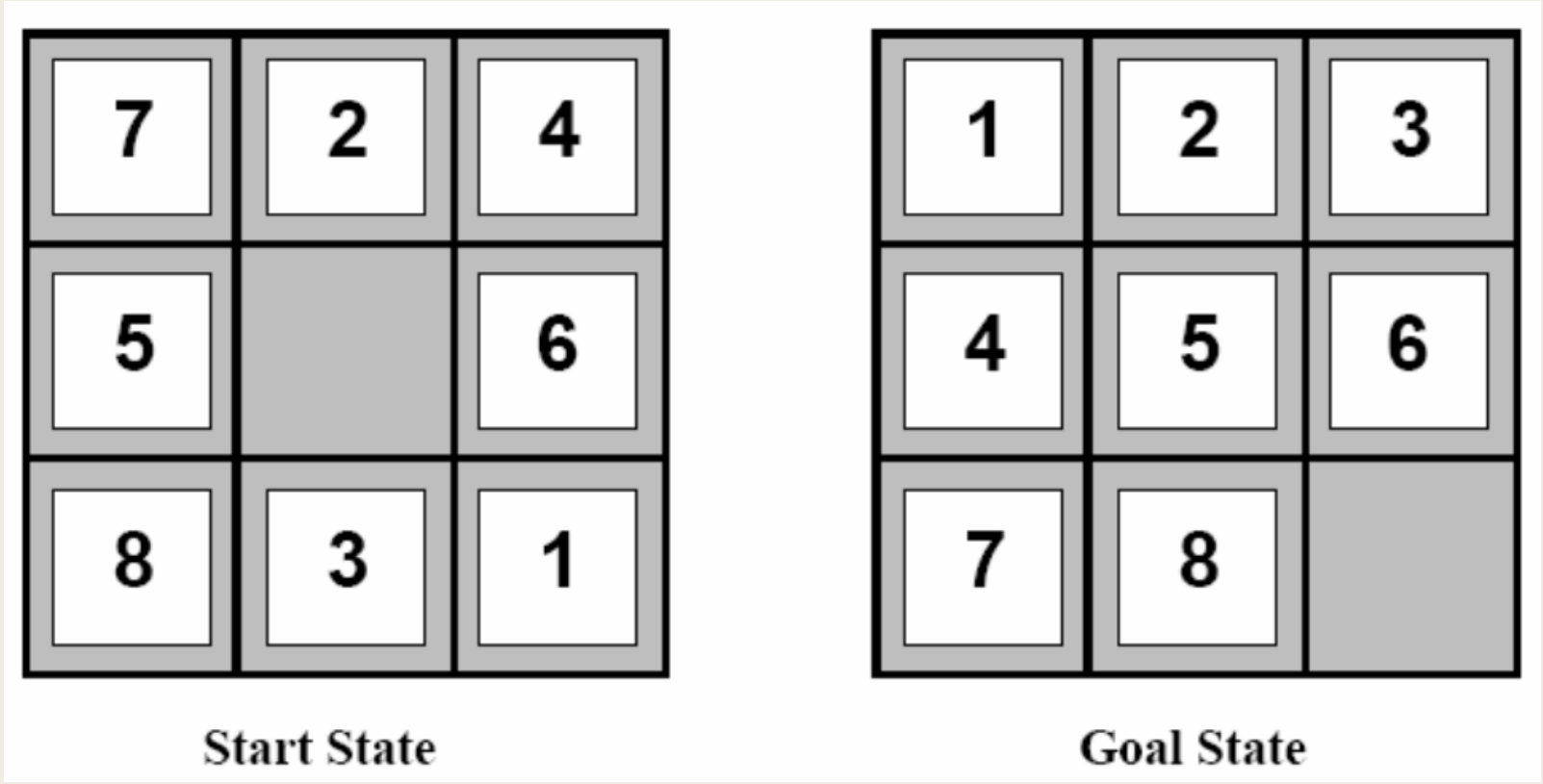
Problem Solution

- A **path** in the state space is a sequence of states connected by a sequence of actions.
- A **solution** in the state space is a path from the initial state to a goal state or, sometimes, just a goal state.
- **Path/solution cost:** function that assigns a numeric cost to each path.
- Solution quality is measured by the path cost function, and an **optimal solution** has the lowest path cost among all solutions.

Problem Description

- Components:
  - State space
  - Initial state
  - Goal state (or the conditions it has to fulfill)
  - Available actions (operators to change state)
  - Restrictions (e.g., cost)

Example: 8-puzzle



Example: 8-puzzle

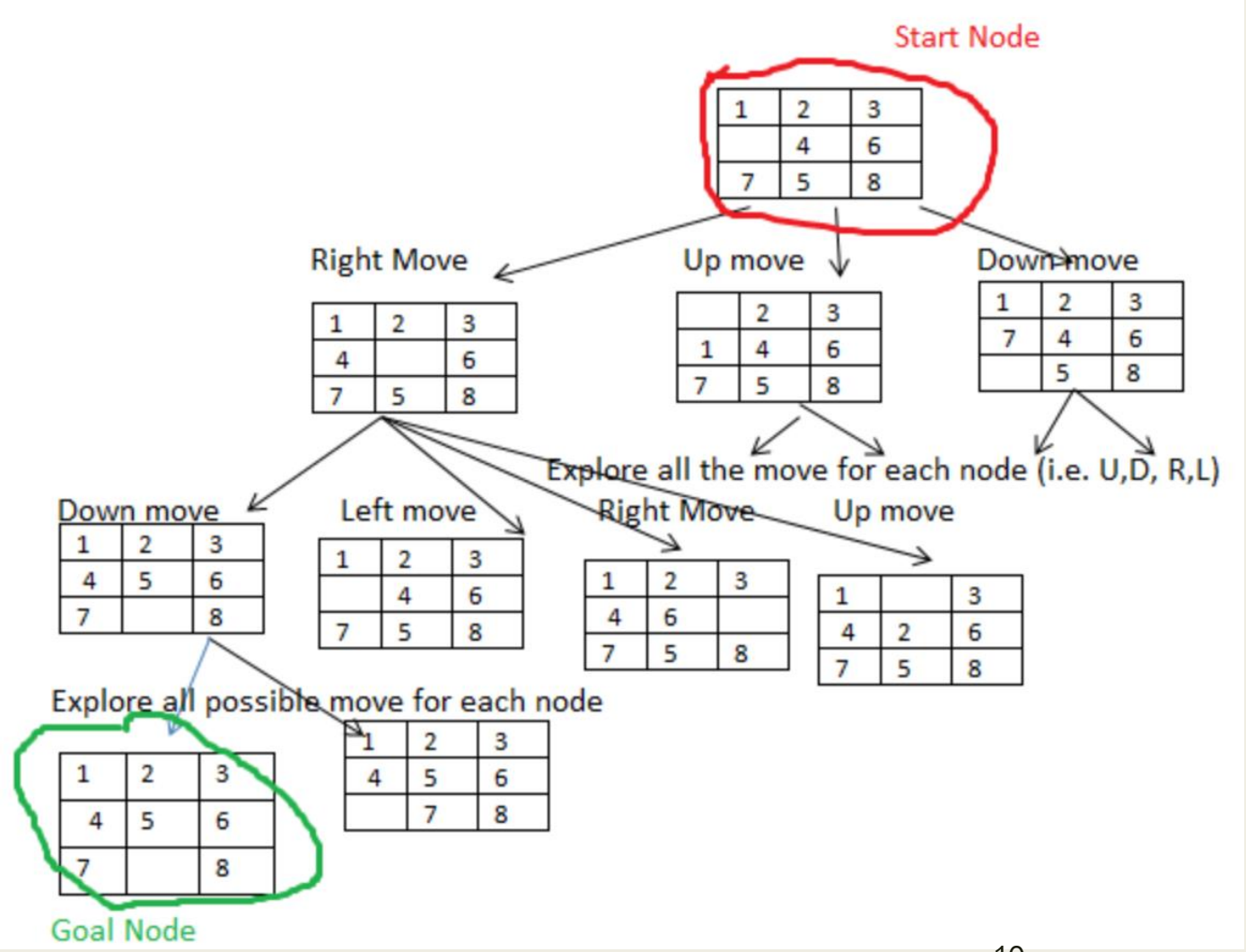
- **State space:** configuration of the eight tiles on the board
- **Initial state:** any configuration
- **Goal state:** tiles in a specific order
- **Operators or actions:** "blank moves"

– Condition: the move is within the board

– Transformation: blank moves *Left*, *Right*, *Up*, or *Down*

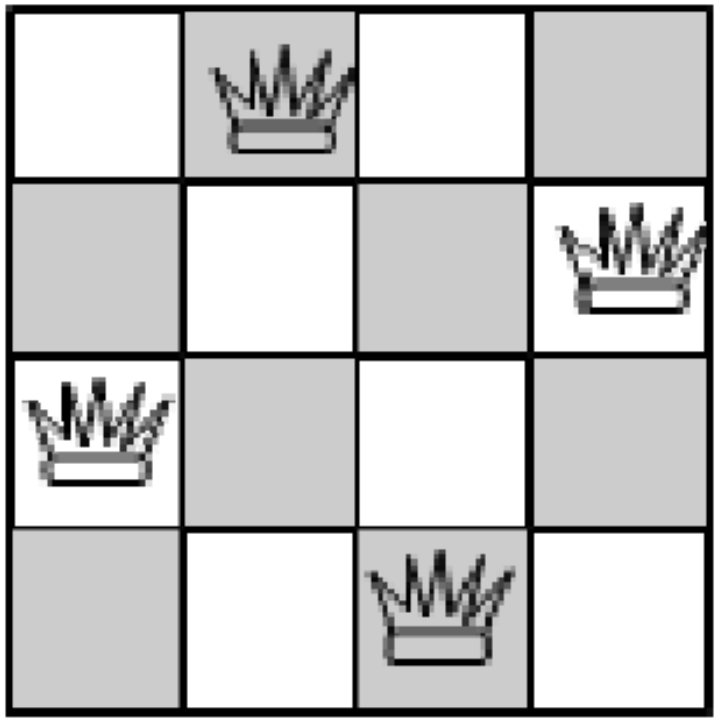
- **Solution** will be a sequence of moves of the blank that transform the initial state to goal state.

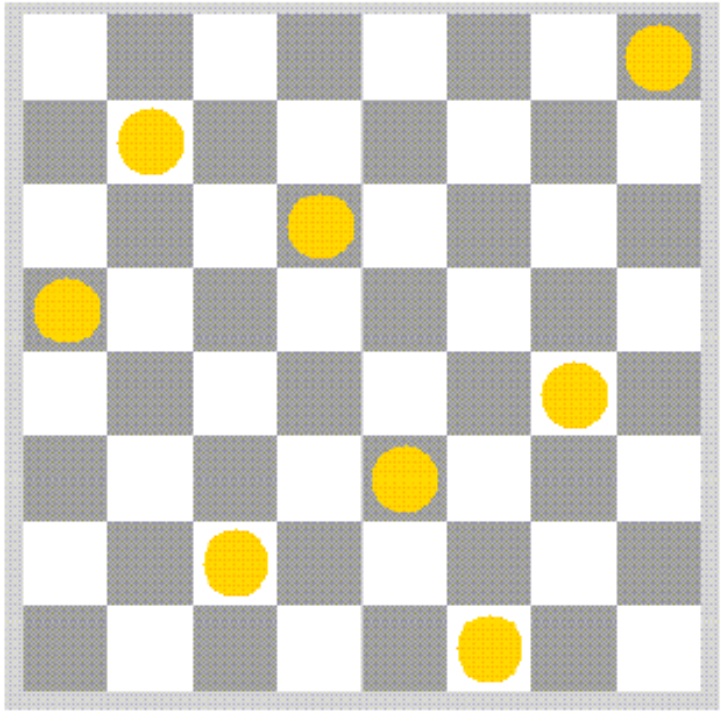
Here, our goal condition is satisfied by only a single state.



Example: n-queens

$(n = 4, n = 8)$





Example: n-queens

- **State space**: configurations from 0 to *n* queens on the board with only one queen per row and column
- **Initial state**: configuration without queens on the board
- **Goal state**: configuration with *n* queens such that no queen attacks any other
- **Operators or actions**: place a queen on the board

  - Condition: the new queen is not attacked by any other already placed

  - Transformation: place a new queen in a particular square of the board
- **Solution**: solution after fulfilling the criteria.

Structure of the state space

- Data structures:

– Trees: only one path to a given node

– Graphs: several paths to a given node

- Operators: directed arcs between nodes
- The search process explores the state space.
- In the worst case all possible paths between the initial state and the goal state are explored.

To build a system to solve a problem

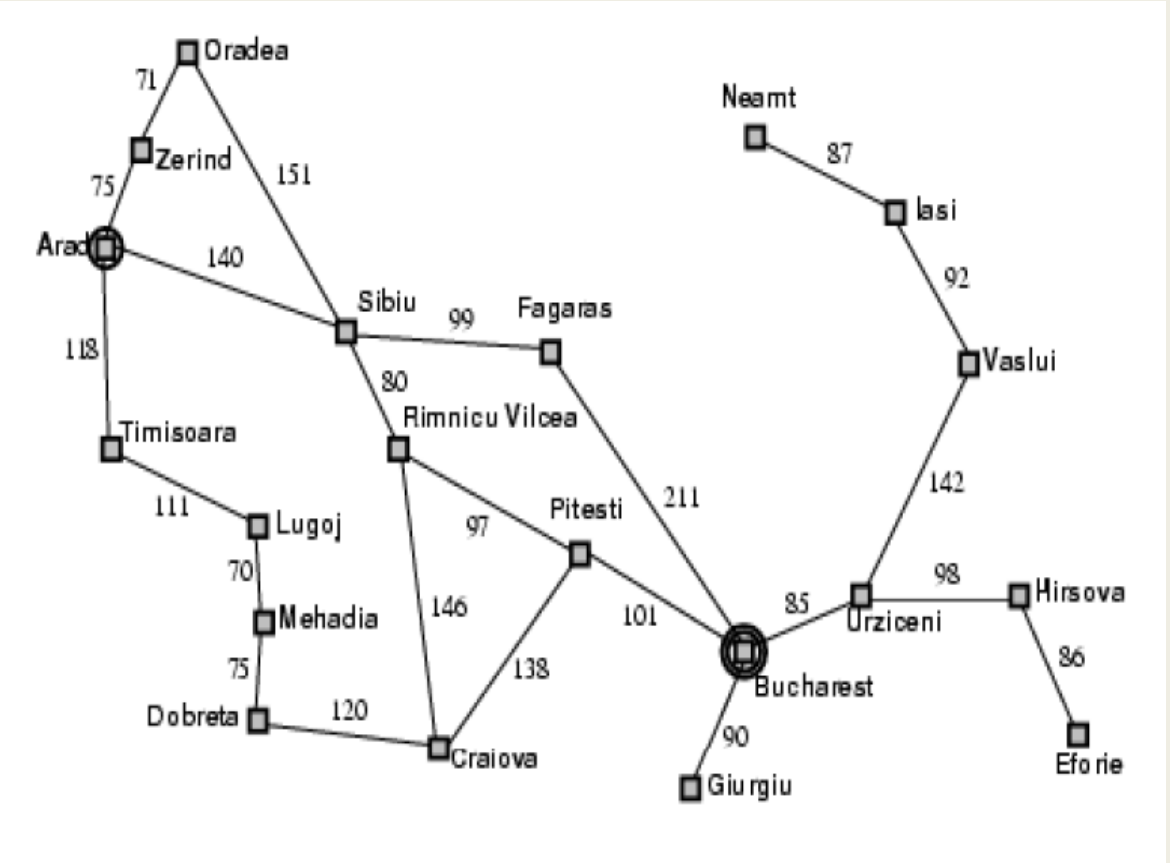
1. Define the problem precisely
2. Analyze the problem
3. Isolate and represent the task knowledge that is necessary to solve the problem
4. Choose the best problem-solving techniques and apply it to the particular problem.

Defining the problem as State Space Search

- Search is a very important process in the solution of hard problems for which no more direct techniques are available.

Example: Romania

- On holiday in Romania; currently in Arad.
- Flight leaves tomorrow from Bucharest
- Formulate goal:
  - be in Bucharest
- Formulate problem:
  - states: various cities
  - actions: drive between cities
- Find solution:
  - sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

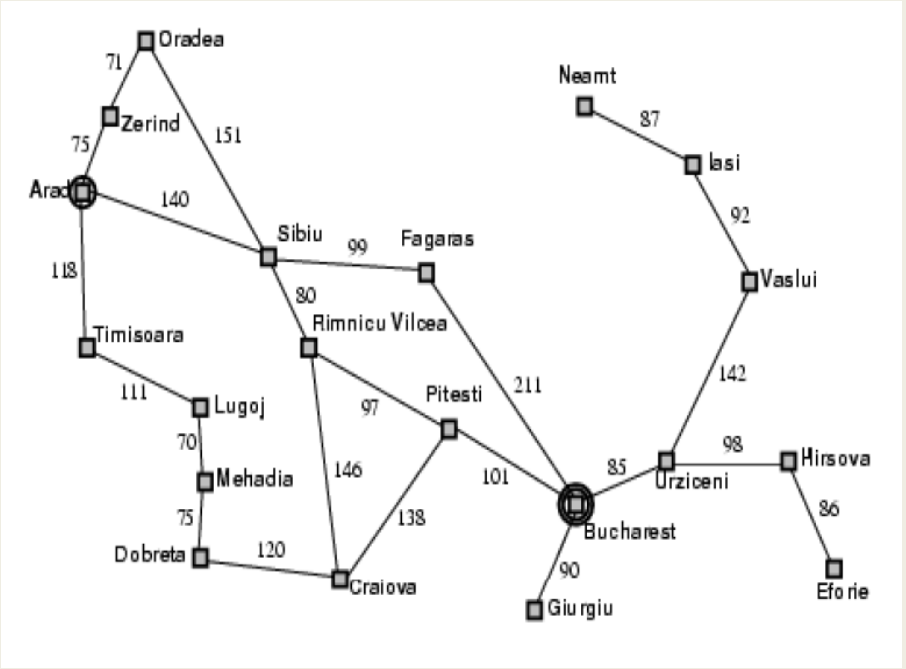


State-Space : Problem Formulation

A **problem** is defined by four items:

- **initial state** e.g., "at Arad“
- **actions** or successor function *: Given a particular state s, action(s) returns* the set of actions is applicable in s. For example, from the state In(Arad), the applicable actions are {Go(sibiu), Go(Timisoara), GO(Zerind)}
- **goal test**, (or goal state)

  e.g., *x* = "at Bucharest”, *Checkmate(x)*
- **path cost** (additive)
  - e.g., sum of distances, number of actions executed, etc.
  - *c(x,a,y)* is the step cost, assumed to be ≥ 0
- A **solution** is a sequence of actions leading from the initial state to a goal state.

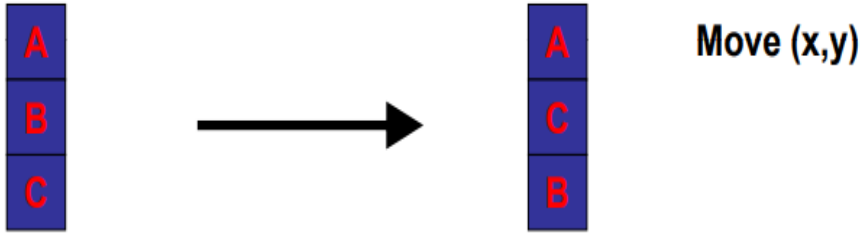


Defining Search Problems

- **A statement of a Search problem has 4 components**
  - 1. A set of states
  - 2. A set of “operators” which allow one to get from one state to another
  - 3. A start state S
  - 4. A set of possible goal states, or ways to test for goal states
  - 4a. Cost path
- **A solution consists of**
  - a sequence of operators which transform S into a goal state G

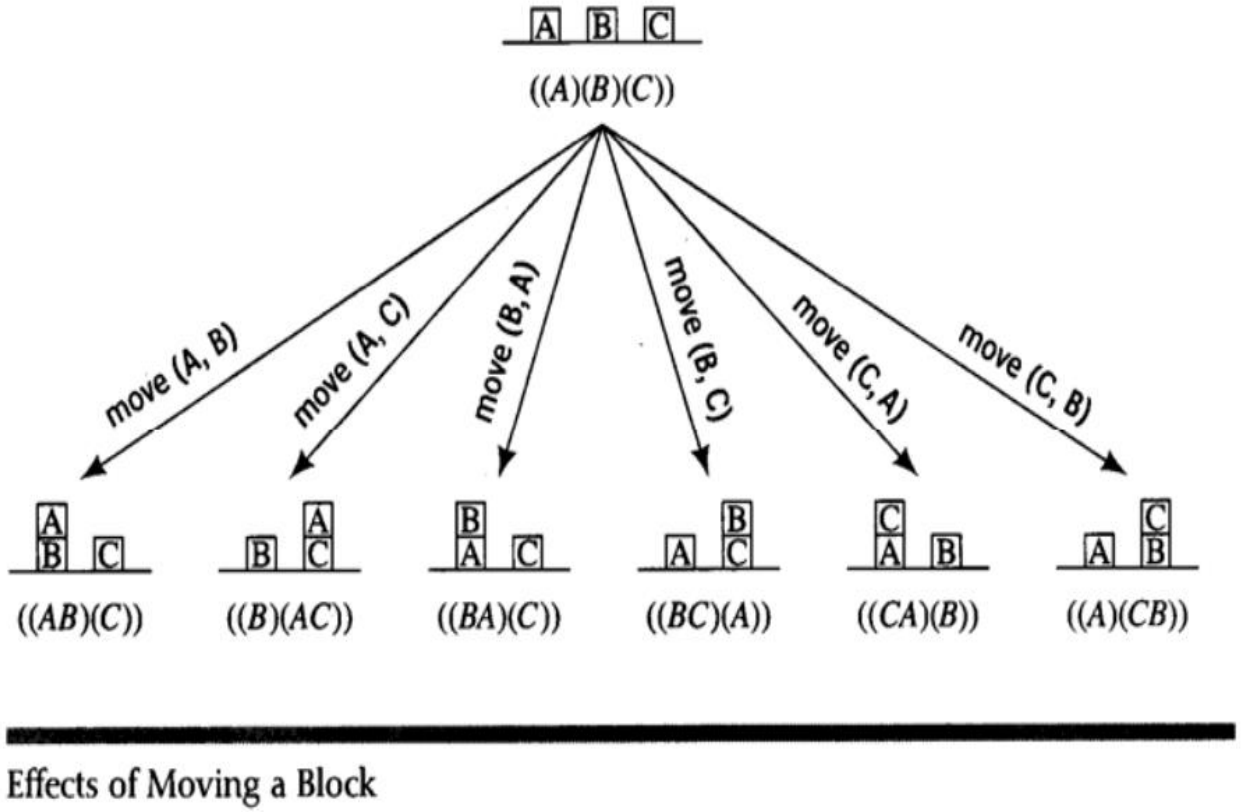
Robot block world

- Given a set of blocks in a certain configuration,
- Move the blocks into a goal configuration.
- Example :
  - (c,b,a) → (b,c,a)



Move (x,y)

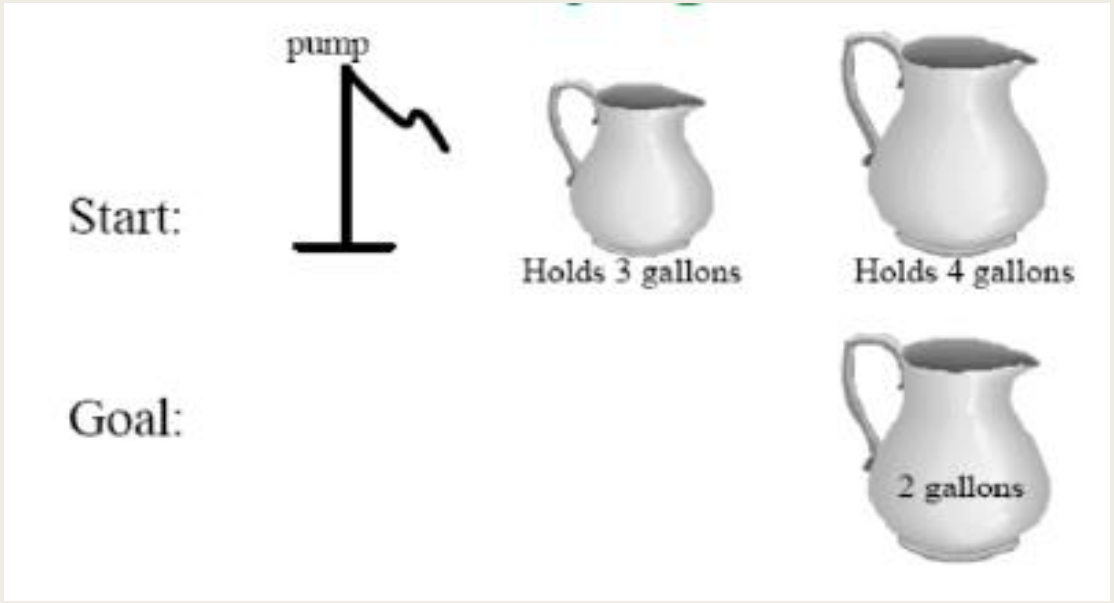
Operator Description



Effects of Moving a Block

Water Jug Problem

- You are given two jugs, a 4-litre one and a 3-litre one.
- Neither has any measuring markers on it.
- There is a pump that can be used to fill the jugs with water.
- How can you get exactly 2 liters of water into 4-litre jug.



State Space Search: Water Jug Problem

- State: (x, y) where,

  x = 0, 1, 2, 3, or 4

  y = 0, 1, 2, 3
- Start state: (0, 0).
- Goal state: (2, n) for any n.
- Attempting to end up in a goal state.

Production Rules for Water Jug Problem

Sl No	Current state	Next State	Descritpion
1	(x,y) if x < 4	(4,y)	Fill the 4 gallon jug
2	(x,y) if y <3	(x,3)	Fill the 3 gallon jug
3	(x,y) if x > 0	(x-d, y)	Pour some water out of the 4 gallon jug
4	(x,y) if y > 0	(x, y-d)	Pour some water out of the 3-gallon jug
5	(x,y) if x>0	(0, y)	Empty the 4 gallon jug
6	(x,y) if y >0	(x,0)	Empty the 3 gallon jug on the ground
7	(x,y) if x+y >= 4 and y >0	(4, y-(4-x))	Pour water from the 3 – gallon jug into the 4 –gallon jug until the 4-gallon jug is full

Production Rules for Water Jug Problem

8	(x, y) if x+y >= 3 and x>0	(x-(3-y), 3)	Pour water from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full
9	(x, y) if x+y <=4 and y>0	(x+y, 0)	Pour all the water from the 3-gallon jug into the 4-gallon jug
10	(x, y) if x+y <= 3 and x>0	(0, x+y)	Pour all the water from the 4-gallon jug into the 3-gallon jug
11	(0,2)	(2,0)	Pour the 2 gallons from 3-gallon jug into the 4-gallon jug
12	(2,y)	(0,y)	Empty the 2 gallons in the 4-gallon jug on the ground

State Space Search: Water Jug Problem

1. Current state = (0, 0)
2. Loop until reaching the goal state (2, 0)
    - Apply a rule whose left side matches the current state
    - Set the new current state to be the resulting state

        (0, 0)

        (0, 3)

        (3, 0)

        (3, 3)

        (4, 2)

        (0, 2)

        (2, 0)

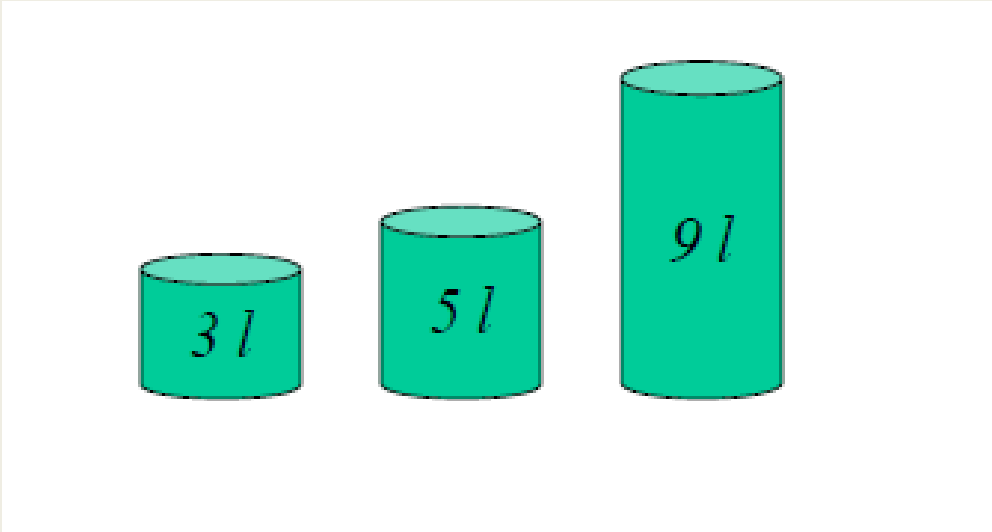
To Solve Water Jug Problem

- Required a control structure that loops through a simple cycle in which some rule whose left side matches the current state is chosen the appropriate change to the state is made as described in the corresponding right side, and the resulting state is checked to see if it corresponds to goal state.

Gallons in the 4-gallon jug	Gallons in the 3-gallon jug	Rule applied
0	0	-
0	3	2
3	0	9
3	3	2
4	2	7
0	2	5 or 12
2	0	9 or 11

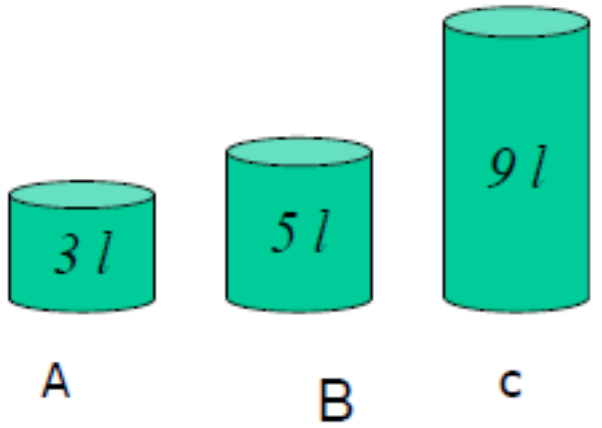
Try it: Measuring Problem– Water Jug Problem!

- **Problem:** Using these three buckets, measure 7 liters of water.



Measuring Problem!

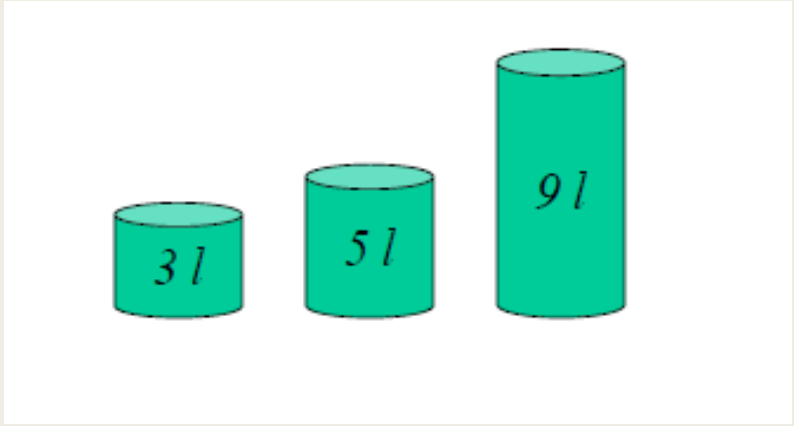
A	B	C
0	0	0
3	0	0
3	0	3
0	0	6
3	0	6
0	3	6
3	3	6
1	5	6
0	5	7



Measuring Problem!

- **Another Solution:**

A	B	C	
0	0	0	start
0	5	0	
3	2	0	
3	0	2	
3	5	2	
3	**0**	**7**	**goal**

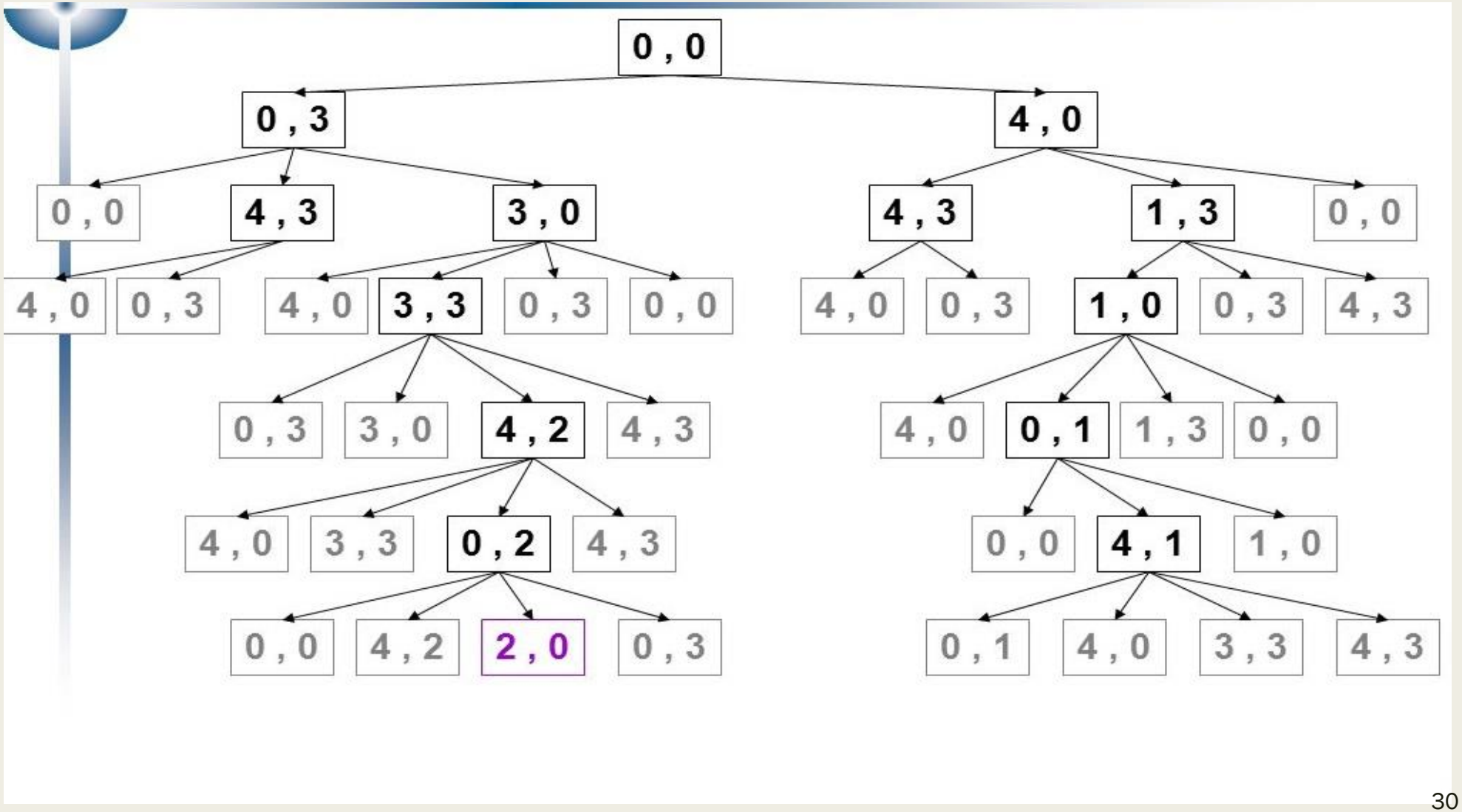


State Space Tree

- State space tree is a tree constructed from all of the possible states of the problem as nodes, connected via state transitions from some initial state as root to some terminated state as leaf.

- If the entire state space representations for a problem is given, it is possible to trace the path from initial state to the goal state and identify the sequence of operators necessary for doing it.

State Space Tree for Water-Jug Problem



Sum of subsets

- **Problem:**

Given *n* positive integers $w_1, \ldots w_n$ and a positive integer S. Find all subsets of $w_1, \ldots w_n$ that sum to S.

- **Example:**

n=3, S=6, and w_1=2, w_2=4, w_3=6

- **Solutions:**

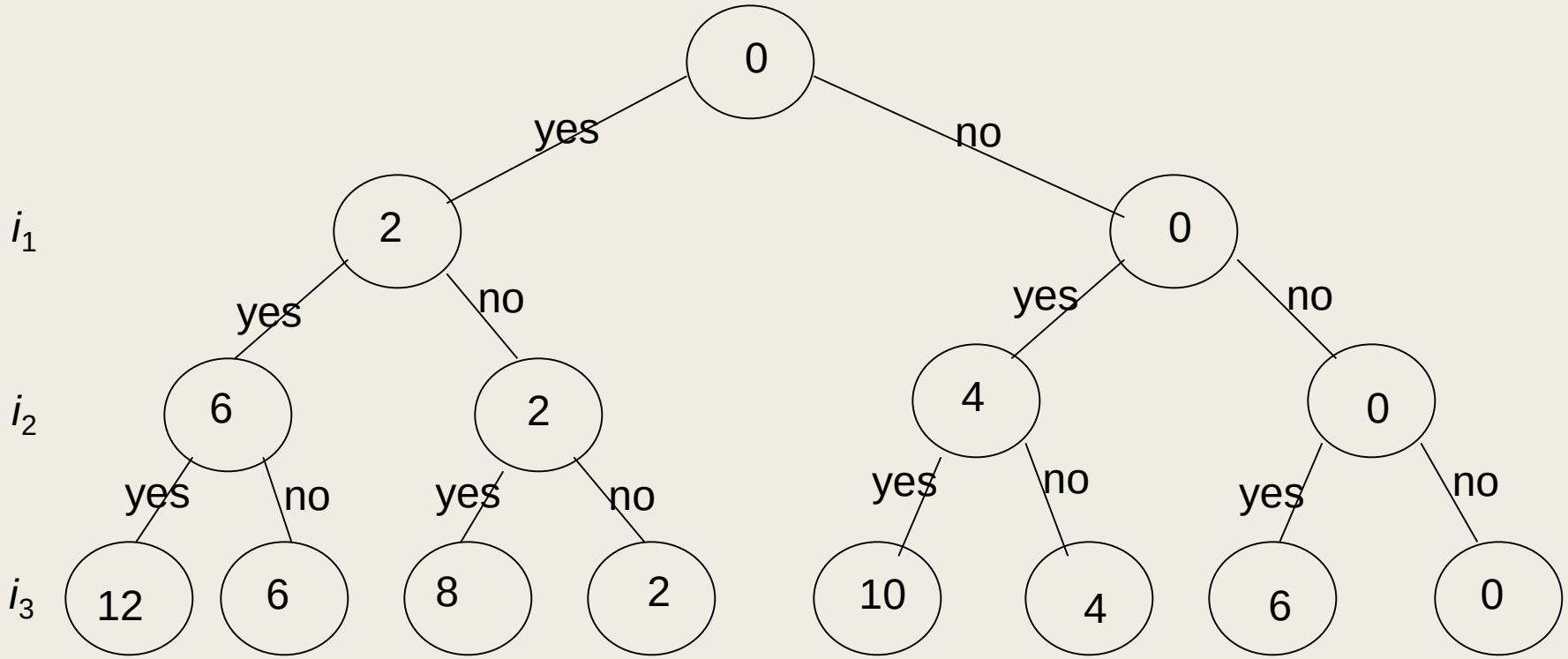
{2,4} and {6}

Sum of subsets

- We will assume a binary state space tree.
- The nodes at depth 1 are for including (yes, no) item 1, the nodes at depth 2 are for item 2, etc.
- The left branch includes w_i, and the right branch excludes w_i.
- The nodes contain the sum of the weights included so far.

Sum of subset Problem: State Space Tree

$w_1 = 2,\ w_2 = 4,\ w_3 = 6$ and $S = 6$



The sum of the included integers is stored at the node.

Backtracking

- **Definition:** We call a node *nonpromising* if it cannot lead to a feasible (or optimal) solution, otherwise it is *promising.*

- **Main idea:** Backtracking consists of doing a DFS of the state space tree, checking whether each node is promising and if the node is non-promising, then backtrack to the node's parent.

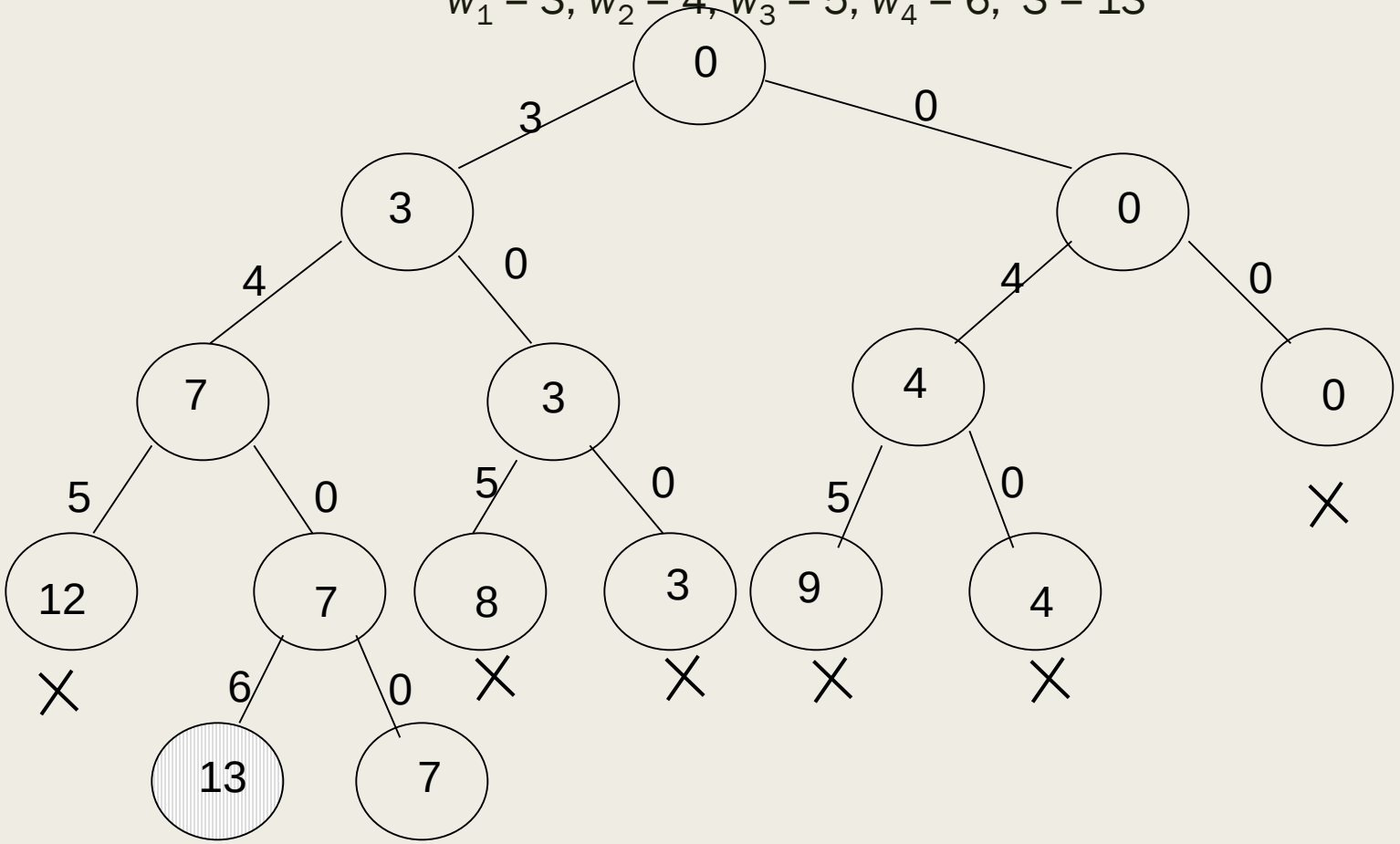
Backtracking

- The state space tree consisting of expanded nodes only is called the *pruned state space tree.*
- The following slide shows the pruned state space tree for the sum of subsets example.
- There are only 15 nodes in the pruned state space tree.
- The full state space tree has 31 nodes.

A Pruned State Space Tree

(find all solutions)

$w_1 = 3, w_2 = 4, w_3 = 5, w_4 = 6;\ S = 13$



Sum of subsets problem

Farmer River Crossing Problem

- A farmer with his wolf, goat and cabbage come to the edge of a river. They all want to cross the river. There is a boat at the river's edge, but only the farmer can row, the boat can carry two things at a time. If the wolf is ever left alone with the goat, the wolf will eat the goat. Similarly, if the goat is left alone with the cabbage, the goat will eat the cabbage.

Solution

- A set of all possible states for a problem is known as the state space of the problem. There are 16 sub-states of the man , wolf, goat and cabbage.
- Some of the 16 states such as GC-MW are fatal and may never be entered by the system.

MWGC-ϕ → initial state

ϕ –MWGC → goal state

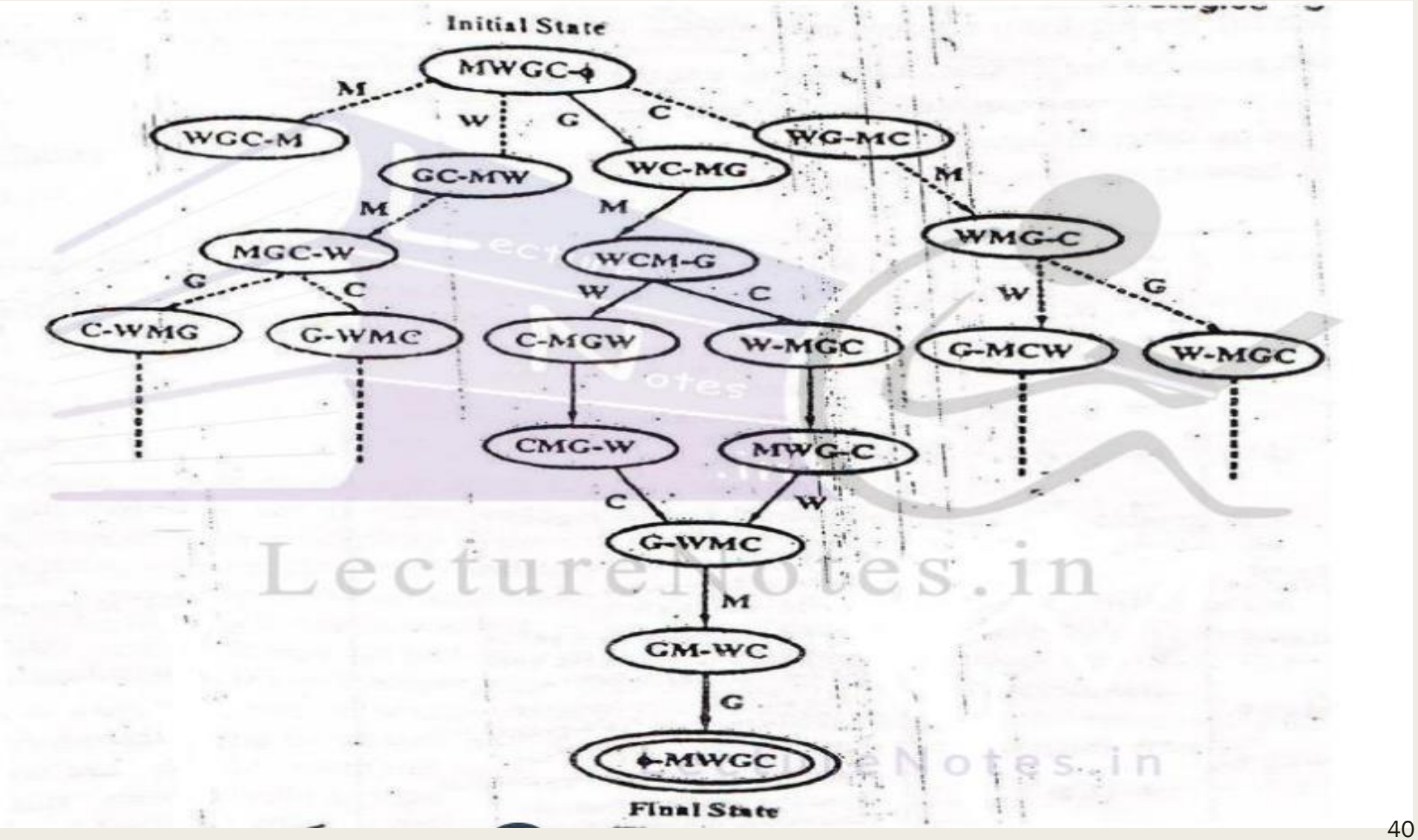
LectureNotes.in

Sequence of Steps

1. First man crosses the river with goat.
2. Man comes back.
3. Man takes either cabbage (or wolf) with him to another side.
4. Man comes back with goat to first side.
5. Man takes wolf (or cabbage) with him to another side.
6. Man comes back.
7. Man takes goat with him to another side.

LectureNotes.in

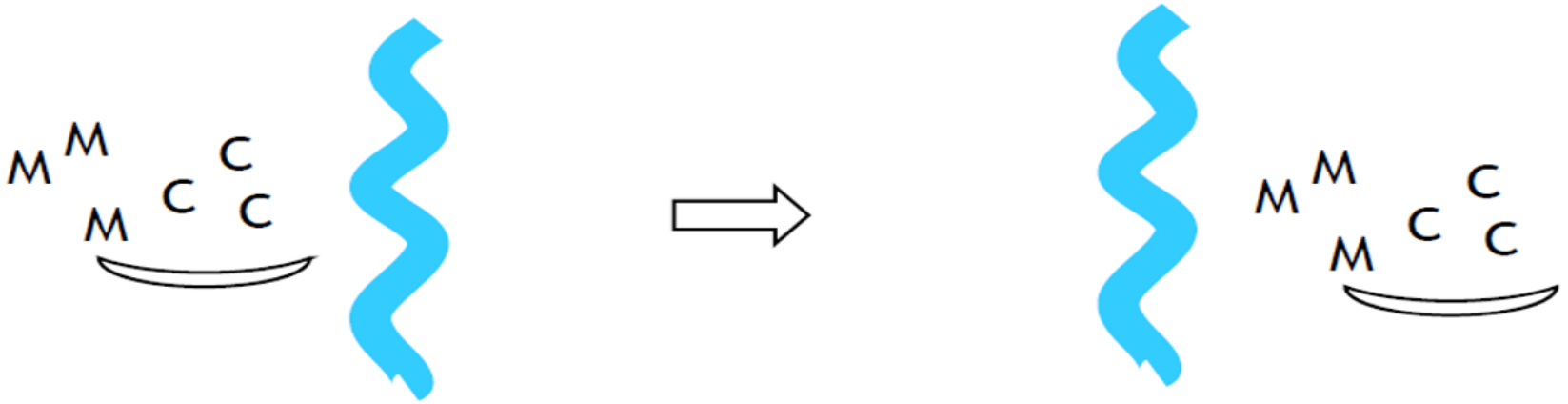
State Space Representation



LectureNotes.in

Missionaries and Cannibals Problem

3 Missionaries and 3 Cannibals wish to cross the river. They have a boat that will carry two people. Everyone can navigatethe boat. If at any time the Cannibals outnumber the missionaries on either bank of the river, they will eat the Missionaries. Find the smallest number of crossings that will allow everyone to cross the river safely.
The problem can be solved in 11 moves. But people rarely get the optimal solution, because the MC problem contains a 'tricky' state at the end, where two people move back across the river.

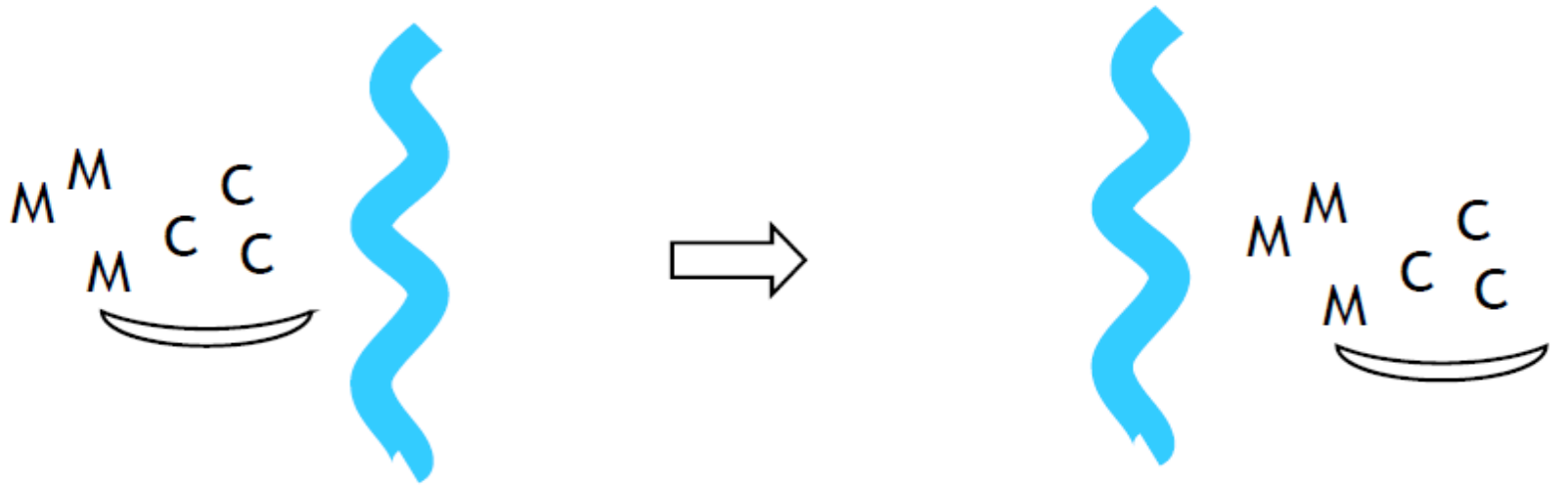


Initial state

Goal state

Missionaries and Cannibals Problem

- An old puzzle is the “Missionaries and cannibals” problem (in various guises)
- The missionaries and cannibals wish to cross a river
- They have a canoe that can hold two people
- It is unsafe to have cannibals outnumber missionaries



Initial state

Goal state

Missionaries and Cannibals Problem

Step	Left	Boat	Right
0	MMM CCC		
1	MMMC	CC ->	CC
2	MMMCC	<- C	C
3	MMM	CC ->	CCC
4	MMMC	<- C	CC
5	MC	MM ->	MM CC
6	MMCC	<- MC	M C
7	CC	MM ->	MMM C
8	CCC	<- C	MMM
9	C	CC ->	MMM CC
10	CC	<- C	MMM C
11		CC ->	MMM CCC

M: Missionary
C: Cannibal

Missionaries and Cannibals Problem

